Note complémentaire SQL n°1

Ex æquo et LIMIT, jointures multiples | 16 octobre 2026

1. Ex æquo : le piège de LIMIT

`ORDER BY ... LIMIT n` coupe le résultat après n lignes, même si plusieurs lignes ont la même valeur à la frontière : la ligne gardée est alors choisie sans règle claire et peut changer d'une exécution à l'autre. Il faut décider ce que demande la question métier : « exactement n lignes » ou « tous ceux qui sont dans le top n » ?

```
-- Exactement 3 lignes, résultat stable : second critère de tri
SELECT * FROM commandes ORDER BY montant, id LIMIT 3;

-- Tous les ex aequo du top 3 : RANK dans un CTE
WITH classement AS (
    SELECT *, RANK() OVER (ORDER BY montant) AS rang
    FROM commandes
)
SELECT * FROM classement WHERE rang <= 3;

-- La ou les villes à l'âge moyen maximal (même principe que WHERE age = MIN)
WITH moy AS (
    SELECT ville, AVG(age) AS age_moyen FROM clients GROUP BY ville
)
SELECT ville, age_moyen
FROM moy
WHERE age_moyen = (SELECT MAX(age_moyen) FROM moy);
```

2. Mise en place : deux nouvelles tables

Un catalogue `produits` (avec une catégorie) relié à `commandes` par `produit_id`, et une table `villes` (avec la province) reliée à `clients` par le nom de la ville.

```
SET SQL_SAFE_UPDATES = 0;

CREATE TABLE produits (id INT PRIMARY KEY, nom VARCHAR(50), categorie VARCHAR(50));
INSERT INTO produits VALUES (1, 'Telephone', 'Electronique');
INSERT INTO produits VALUES (2, 'Casque', 'Accessoires');
INSERT INTO produits VALUES (3, 'Ordinateur', 'Electronique');
INSERT INTO produits VALUES (4, 'Souris', 'Accessoires');
INSERT INTO produits VALUES (5, 'Clavier', 'Accessoires');
INSERT INTO produits VALUES (6, 'Tablette', 'Electronique');
INSERT INTO produits VALUES (7, 'Montre', 'Accessoires');

ALTER TABLE commandes ADD COLUMN produit_id INT;

UPDATE commandes o
INNER JOIN produits p ON o.produit = p.nom
SET o.produit_id = p.id;

CREATE TABLE villes (ville VARCHAR(50), province VARCHAR(50));
INSERT INTO villes VALUES ('Kinshasa', 'Kinshasa');
INSERT INTO villes VALUES ('Lubumbashi', 'Haut-Katanga');
INSERT INTO villes VALUES ('Goma', 'Nord-Kivu');
```

3. Jointures multiples : une jointure à la fois, en chaîne

Chaque nouveau JOIN relie une table à une table **déjà présente** dans la requête.

```
-- 3 tables : prénom, produit, catégorie, montant (6 lignes)
SELECT c.prenom, p.nom AS produit, p.categorie, o.montant
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id
INNER JOIN produits p ON o.produit_id = p.id;

-- 4 tables : total par province et par catégorie
-- Kinshasa/Accessoires 85, Kinshasa/Electronique 550, Haut-Katanga/Electronique 800
SELECT v.province, p.categorie, SUM(o.montant) AS total
FROM clients c
INNER JOIN villes v ON c.ville = v.ville
INNER JOIN commandes o ON c.id = o.client_id
INNER JOIN produits p ON o.produit_id = p.id
GROUP BY v.province, p.categorie;
```

4. Les trois précautions

Précaution	Pourquoi
Construire par étapes	Lancer avec 2 tables, vérifier, puis ajouter la 3e, etc.
Compter les lignes à chaque étape	Une jointure un-à-plusieurs multiplie les lignes : un SUM peut compter deux fois le même m
Ne pas mélanger LEFT et INNER sans réfléchir	Un LEFT JOIN commandes puis INNER JOIN produits fait disparaître les clients sans co

5. Exercices bonus et corrections (partie 8)

```
-- 8.1 Montant total par catégorie : Electronique 1350, Accessoires 205
SELECT p.categorie, SUM(o.montant) AS total
FROM produits p
INNER JOIN commandes o ON p.id = o.produit_id
GROUP BY p.categorie;

-- 8.2 Montant total par province : Kinshasa 635, Haut-Katanga 800
SELECT v.province, SUM(o.montant) AS total
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id
INNER JOIN villes v ON c.ville = v.ville
GROUP BY v.province;

-- 8.3 Détail des commandes de clients connus (4 tables, 6 lignes)
SELECT c.prenom, v.province, p.nom AS produit, o.montant
FROM clients c
INNER JOIN villes v ON c.ville = v.ville
INNER JOIN commandes o ON c.id = o.client_id
INNER JOIN produits p ON p.id = o.produit_id;

-- 8.4 Tous les clients avec province et nombre de commandes (8 lignes)
SELECT c.id, c.prenom, v.province, COUNT(o.id) AS nb_commandes
FROM clients c
LEFT JOIN villes v ON c.ville = v.ville
LEFT JOIN commandes o ON c.id = o.client_id
```

```
GROUP BY c.id, c.prenom, v.province;

-- 8.5 Catégorie la plus vendue par province
-- Résultat : Kinshasa / Electronique 550, Haut-Katanga / Electronique 800
WITH classement AS (
    SELECT v.province, p.categorie, SUM(o.montant) AS total_vendu
    FROM villes v
    INNER JOIN clients c ON v.ville = c.ville
    INNER JOIN commandes o ON c.id = o.client_id
    INNER JOIN produits p ON p.id = o.produit_id
    GROUP BY v.province, p.categorie
),
classement_final AS (
    SELECT province, categorie, total_vendu,
           RANK() OVER (PARTITION BY province ORDER BY total_vendu DESC) AS rang
    FROM classement
)
SELECT * FROM classement_final WHERE rang = 1;
```

6. L'erreur à retenir : une condition ON qui ne relie rien

Dans une première version du 8.5, la jointure était écrite `INNER JOIN produits b ON c.id = c.produit_id` : les deux côtés de la condition viennent de la même table (commandes), et aucun ne vient de produits. Comme les identifiants des commandes coïncidaient ici avec ceux des produits, la condition était toujours vraie : chaque commande a été associée aux 7 produits (3 électroniques et 4 accessoires). Les montants ont été multipliés par 3 et par 4, et le résultat affichait **Accessoires** (3200 et 2540) au lieu de **Electronique** (800 et 550).

Ce résultat avait l'air plausible : seul un contrôle l'aurait révélé. Un total par groupe (3200) ne peut pas dépasser le total de toutes les commandes (1555).

7. À retenir

• Avec LIMIT, toujours se demander : que se passe-t-il en cas d'égalité ? Ajouter un second critère de tri, ou utiliser RANK pour garder les ex æquo.

• Comparer à la valeur extrême (WHERE x = (SELECT MAX(x) ...)) garde tous les ex æquo, contrairement à ORDER BY ... LIMIT 1.

• Plusieurs tables = plusieurs JOIN enchaînés, chacun avec son propre ON.

• Toujours vérifier le nombre de lignes après chaque jointure ajoutée.

• Chaque condition ON doit relier la table ajoutée à une table déjà présente (`p.id = o.produit_id`), jamais deux colonnes d'une même table.

• Contrôle de cohérence : un total par groupe ne peut pas dépasser le total global, et la somme des groupes doit correspondre au total attendu.

• Des alias parlants (c pour clients, o pour commandes, p pour produits, v pour villes) évitent ce genre de confusion, contrairement à a, b, c, d.

• Sous MySQL, UPDATE ... JOIN permet de remplir une colonne à partir d'une autre table.